DUALE HOCHSCHULE SACHSEN

Verteilte Systeme

Wintersemester 2026

Dr. Michael Hofmann

DHSN: Glauchau

14.10.2026

Allgemeine Informationen

Ablauf

- 6 Termine mit je 2x2 LVS (2x90 Minuten) = 24 LVS

⇝ jeweils „Vorlesung“ + Beispiele und ggf. Übungsaufgaben

- Vorlesungsmaterialen ⇝ OPAL: Theorie der Betriebssysteme und verteilten Systeme
- Abschlussklausur: 120 Minuten ⇝ $\frac{1}{3}$ für Verteilte Systeme

Literatur

- Andrew S. Tanenbaum und Maarten van Steen. *Verteilte Systeme: Grundlagen und Paradigmen.* 2. Ausgabe, Addison Wesley, 2007.

Themenüberblick (aktueller Plan)

(1) 31.08.2026
- Einführung

(2) 10.09.2026
- Architekturen

(3) 14.09.2026
- Prozesse

(4) 14.10.2026
- Kommunikation

(5) 27.10.2026
- Synchronisation

(6) 02.11.2026
- Konsistenz und Replikation
- Fehlertoleranz
- Wiederholung, Fragen & Antworten

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Kommunikation

- Verteilte Systeme erfordern Informationsaustausch zwischen Prozessen ⇝ sog. Interprozesskommunikation (IPC)
- Kommunikation erfolgt durch *primitiven* Nachrichtenaustausch über Netzwerk

Herausforderungen

- Tausende bis Millionen Prozesse
- Netzwerke mit unzuverlässiger Kommunikation (z. B. Internet)

Lösungsansätze

- Protokolle ⇝ Regeln für die Kommunikation
- Kommunikationsmodelle ⇝ Ersatz für *primitiven* Nachrichtenaustausch
  - Remote Procedure Call (RPC)
  - Message-Oriented Middleware (MOM)
  - Datenströme (*Streaming*)
  - Mulitcasting ⇝ Senden an mehrere Empfänger

Inhalt

4. Kommunikation
Protokolle
Remote Procedure Call (RPC)
Nachrichtenorientierte Kommunikation
Datenströme
Multicasting

Protokolle

Welche Aspekte von Kommunikation müssen durch Protokolle geregelt werden?

Protokolle

Welche Aspekte von Kommunikation müssen durch Protokolle geregelt werden?

Beispiele

- Wie viel Volt signalisieren ein 0-Bit oder 1-Bit?
- Wie weiß ein Empfänger was das letzte Bit einer Nachricht ist?
- Wie kann man herausfinden ob eine Nachricht beschädigt oder verloren ist?
- Welches Format haben Zahlen, Strings und andere Datenelemente?

Definition: Protokoll

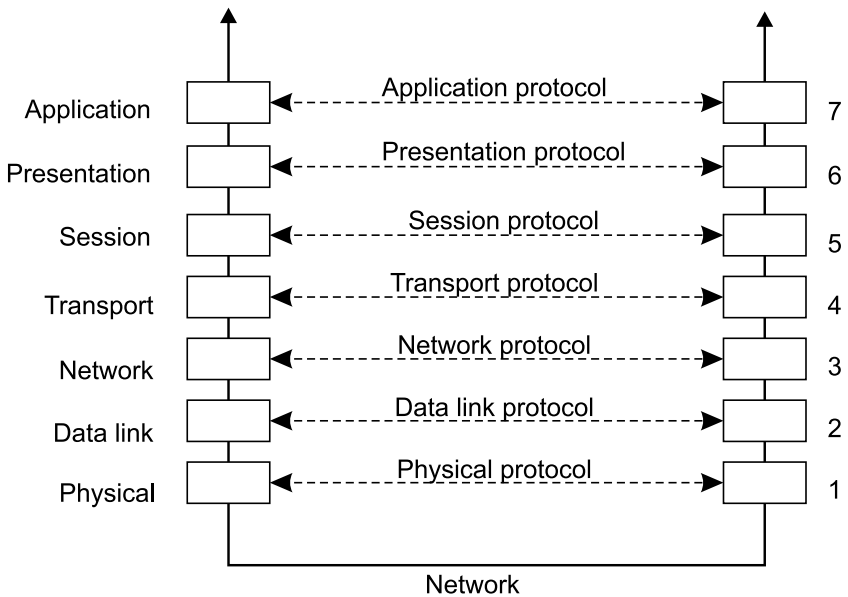
Eine standardisierte Menge von Regeln, die festlegt, wie Daten zwischen Rechnern oder Prozessen ausgetauscht, formatiert und verarbeitet werden.

- Verbindungsorientiert $\rightsquigarrow$ Sender und Empfänger arrangieren eine Verbindung bevor Daten übertragen werden
- Verbindungslos $\rightsquigarrow$ Sender überträgt Daten sobald er bereit dazu ist

Schichten von Protokollen

ISO OSI Model

- Standardisiertes Referenzmodell definiert Ebenen und Verantwortungen
- Schnittstellen mit Operationen zwischen den Schichten



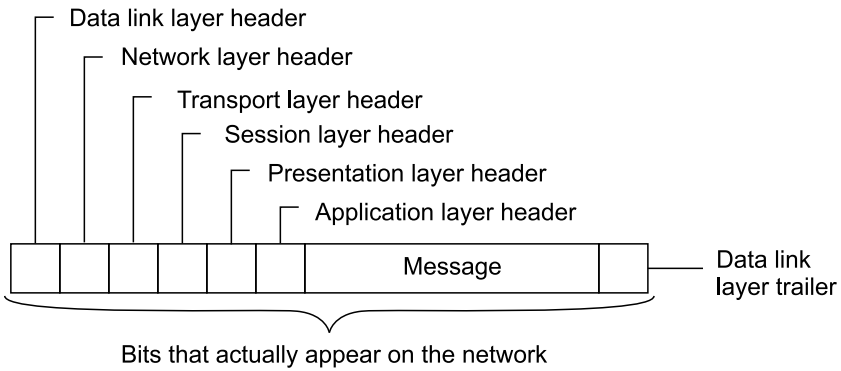
Prozess A sendet Nachricht an B

- Nachricht wird von A an Anwendungsschicht übergeben
- Anwendungsschicht fügt Header hinzu und gibt sie an Darstellungsschicht
- Wiederholung bis Bitübertragungsschicht
- ⇝ Übertragung von A nach B
- Schichten von B analysieren Header und geben Inhalt nach „oben“ weiter

Schichten von Protokollen

ISO OSI Model

- Standardisiertes Referenzmodell definiert Ebenen und Verantwortungen
- Schnittstellen mit Operationen zwischen den Schichten



Prozess A sendet Nachricht an B

- Nachricht wird von A an Anwendungsschicht übergeben
- Anwendungsschicht fügt Header hinzu und gibt sie an Darstellungsschicht
- Wiederholung bis Bitübertragungsschicht

⇝ Übertragung von A nach B

- Schichten von B analysieren Header und geben Inhalt nach „oben“ weiter

Verantwortungen der Protokollschichten

Lower-level Protokolle

- Bitübertragungsschicht ⇝ Übertragung der Bits
- Sicherungsschicht ⇝ Einteilung in Frames, Fehlererkennung mit Prüfsummen
- Vermittlungsschicht ⇝ Wegfindung bzw. Routing, Beispiel: Internet Protocol (IP)

Transportschicht/-protokolle

- Paketierung der Nachrichten ⇝ Reihenfolge
- Integrität der Nachrichten ⇝ Vollständigkeit
- Beispiele: TCP und UDP

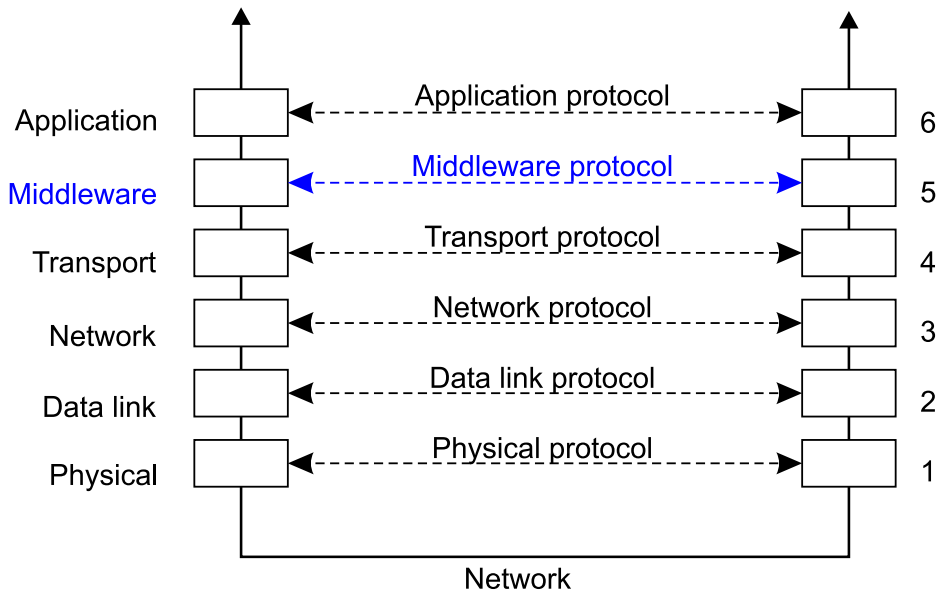
Higher-level Protokolle

- Sitzungsschicht ⇝ Verbesserte Transportschicht
- Darstellungsschicht ⇝ Bedeutung der Bits (z. B. Datenformate)
- Anwendungsschicht ⇝ Konkrete Anwendungen, Beispiele: HTTP und FTP

Middleware-Protokolle

- Middleware ist (hauptsächlich) in der Anwendungsschicht verortet
- Allgemeine Dienste werden über eigene Protokolle bereitgestellt
- Beispiele: Authentifizierung, Autorisierung, Transaktionen, Locks

⇝ Angepasstes Schichtenmodell mit Middleware-Schicht



Arten von Kommunikation

Persistent

- Kommunikationssystem speichert Nachrichten so lange wie nötigt
- ⇝ Sender kann vorzeitig beendet werden

Asynchron

- Sender kann fortfahren
- ⇝ Kommunikationssystem speichert Nachricht

Diskret

- Jede Nachricht ist eine vollständige Informationseinheit

Transient

- Kommunikationssystem speichert Nachrichten so lange Sender und Empfänger ausgeführt werden
- ⇝ Keiner kann vorzeitig beendet werden

Synchron

- Sender blockiert
- ⇝ Verschiedene Synchronisationspunkte möglich

Streaming

- Mehrere, kontinuierliche Nachrichten gehören zusammen

Inhalt

4. Kommunikation
- Protokolle
- Remote Procedure Call (RPC)
- Nachrichtenorientierte Kommunikation
- Datenströme
- Multicasting

Remote Procedure Call (RPC)

- Expliziter Nachrichtenaustausch verhindert Zugriffstransparenz

Grundidee

- Prozess auf Maschine A kann Prozedur auf Machine B aufrufen
- Prozess auf A wird angehalten, während Prozedur auf B ausgeführt wird
- Informationsaustausch zw. Aufrufer (Caller) und Aufgerufenem (Callee)
  - Parameter von Aufrufer an Aufgerufenem
  - Ergebnisse von Aufgerufenem an Aufrufer

Herausforderungen

- Unterschiedliche Hauptspeicher bzw. Adressräume der Maschinen
- Parameter und Ergebnisse müssen ausgetauscht werden
- Unterschiedliche Datenformate auf unterschiedlichen Maschinen
- Maschinen auf beiden Seiten (Caller/Callee) können fehlschlagen

Lokale Prozeduraufrufe

⇝ Prozedur mit bekannter Schnittstelle ist in Programm/Bibliothek vorhanden

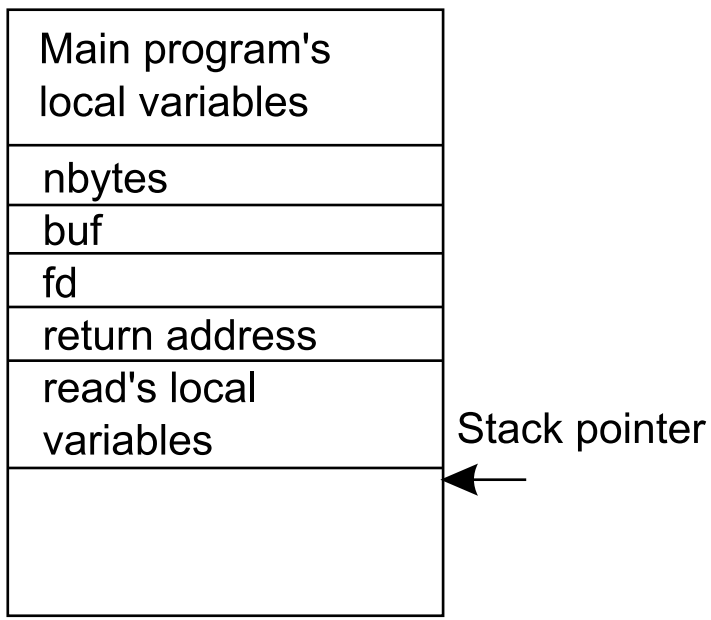
Ablauf

- Programmablauf kommt zu Prozeduraufruf
- Parameter und Rücksprungadresse (nächste Anweisung nach dem Aufruf) werden auf den Stapelspeicher (Stack) gelegt
- Programmablauf springt zu Prozeduradresse
- Parameter werden vom Stack gelesen und verwendet werden
- Bei Prozedurende werden Ergebnisse in Register oder Stack gespeichert
- Programmablauf sprint zu Rücksprungadresse (vom Stack)

Beispiel

```
count = read(fd, buf, nbytes)
```

Stack-Inhalt

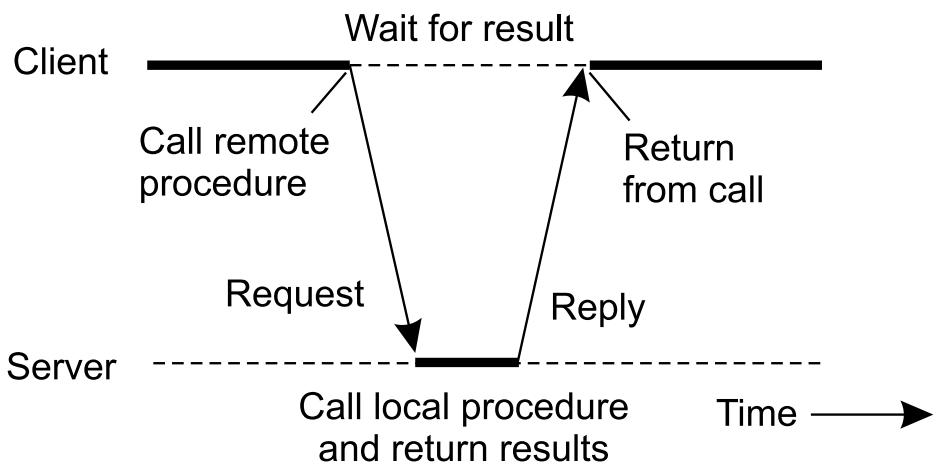


Verteilte Prozeduraufrufe durch Client- und Server-Stubs

Grundidee: Prozess auf Client-Seite ruft Prozedur auf Server-Seite auf

- Client-Stub ist eine Platzhalterprozedur auf Client-Seite mit gleicher Schnittstelle wie Originalprozedur auf Server-Seite
- Platzhalterprozedur verpackt Parameter und sendet sie an Server
- Server-Stub entpackt Parameter, ruft Originalprozedur mit Parametern auf und sendet Ergebnis zurück
- Client-Stub empfängt Ergebnis und gibt es an aufrufenden Prozess zurück

⇝ Regulärer lokaler Prozeduraufruf von Platzhalter- und Originalprozedur



Gesamtablauf eines verteilten Prozeduraufrufs

1. Client-Prozedur ruft Client-Stub wie gewohnt auf.
2. Client-Stub baut Nachricht auf und ruft Betriebssystem auf.
3. Betriebssystem des Clients sendet Nachricht an Betriebssystem des Servers.
4. Betriebssystem des Servers übergibt Nachricht an Server-Stub.
5. Server-Stub packt Parameter aus und ruft Server auf.
6. Server führt Arbeit aus und gibt Ergebnis an Stub zurück.
7. Server-Stub packt Ergebnis in Nachricht und ruft Betriebssystem auf.
8. Betriebssystem des Servers sendet Nachricht an Betriebssystem des Clients.
9. Betriebssystem des Clients übergibt Nachricht an Client-Stub.
10. Stub packt Ergebnis aus und gibt es an Client zurück.

Übergabe von Werte-Parametern

Wie können Werte-Parameter übergeben werden?

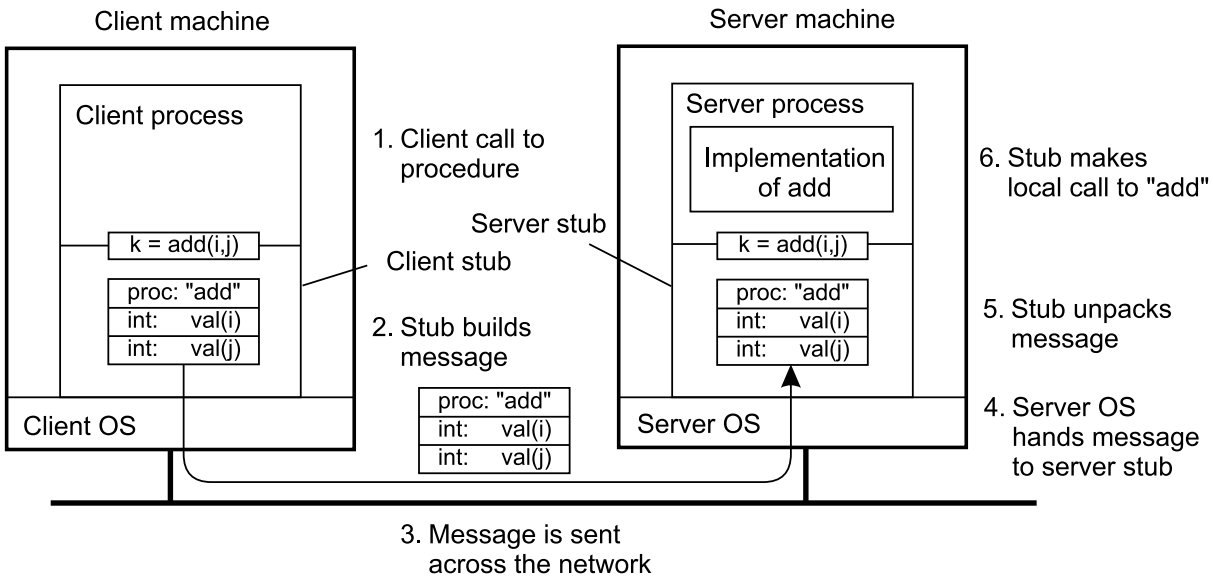
Übergabe von Werte-Parametern

Wie können Werte-Parameter übergeben werden?

- Verpacken der Parameter in eine Nachricht ⇝ *Parameter Marshaling*
- Hinzufügen des Prozedurnamens (zur Prozedurauswahl auf Server)

⇝ Aufbau der Nachricht wird durch RCP-Protokoll definiert

Beispiel: `k = add(i, j)`



Übergabe von Werte-Parametern

Wie können Werte-Parameter übergeben werden?

- Verpacken der Parameter in eine Nachricht $\rightsquigarrow$ *Parameter Marshaling*
- Hinzufügen des Prozedurnamens (zur Prozedurauswahl auf Server)

$\rightsquigarrow$ Aufbau der Nachricht wird durch RCP-Protokoll definiert

Welches Hauptproblem gibt es bei der Übergabe von Werte-Parametern?

Übergabe von Werte-Parametern

Wie können Werte-Parameter übergeben werden?

- Verpacken der Parameter in eine Nachricht $\rightsquigarrow$ *Parameter Marshaling*
- Hinzufügen des Prozedurnamens (zur Prozedurauswahl auf Server)

$\rightsquigarrow$ Aufbau der Nachricht wird durch RCP-Protokoll definiert

Welches Hauptproblem gibt es bei der Übergabe von Werte-Parametern?

Heterogenität in verteilten Systemen

- Unterschiedliche Datentypen, z. B. Bitgrößen und -formate von Integer und Fließkommazahlen
- Unterschiedliche Bitreihenfolgen (*Endianness*) $\rightsquigarrow$ Little vs. Big Endian
- Unterschiedliche Zeichensätze, z. B. ASCII vs. UTF
- Unterschiedliche Programmumgebungen, z. B. Python-Client vs. C-Server
- Lösungsansätze: Plattformunabhängige Formate, Datenserialisierung, ...

Übergabe von Referenz-Parametern

Welches Grundproblem gibt es bei der Übergabe von Referenz-Parametern?

Übergabe von Referenz-Parametern

Welches Grundproblem gibt es bei der Übergabe von Referenz-Parametern?

- Referenzen, d. h. Pointer, sind nur im Adressraum ihres Prozesses gültig

⇝ Pointer-Werte können versendet, aber am Ziel nicht ausgewertet werden

- Pointer können auch in komplexen Datenstrukturen auftreten

Lösungsansätze

- Verbot von Referenzen und Pointern ⇝ nicht (immer) praktikabel
- Explizite Übertragung der referenzierten Werte
  - Einfach möglich für skalare Werte, Arrays und Strukturen
  - Nicht möglich für komplexen Datenstrukturen, z. B. verkette Listen
  - Markierung als Input (konstant) oder Output (veränderlich), um unnötige Übertragungen zu vermeiden

Automatische Generierung von Stub-Code

Welche Informationen werden benötigt um RPC-Stubs zu generieren?

Automatische Generierung von Stub-Code

Welche Informationen werden benötigt um RPC-Stubs zu generieren?

1. Nachrichtenformat und -verarbeitung, Verbindungsdetails ⇝ RCP-Protokoll
2. Konkret verfügbare Prozeduren ⇝ Schnittstellendefinition

RCP-Protokoll

- Wie sind Parameter usw. in der Nachricht angeordnet?
- Wie werden einfache Datentypen (z. B. Integer, Zeichen) dargestellt?
- Wie findet die Netzwerkkommunikation zw. Client und Server statt?

Schnittstellendefinition

- Welchen Namen haben die konkreten Prozeduren?
- Welche Parameter und Ergebnisse inkl. Datentypen haben Prozeduren?

⇝ Beschreibung mittels sog. *Interface Definition Language* (IDL)

Beispiel: Google RPC (gRPC)

- Von Google entwickeltes Open-Source-Framework, das für hohe Leistung und Effizienz ausgelegt ist
- Nutzt HTTP/2-Protokoll für schnellen und effizienten Datenaustausch
- Verwendet *Protocol Buffers* als plattformunabhängige IDL zur effizienten Serialisierung und Definition von Services und Datenstrukturen
- Ermöglicht bidirektionale Vollduplex-Kommunikation, bei der Clients und Server gleichzeitig Daten senden und empfangen können
- Asynchrones Streaming zur effizienten Übertragen großer Datensätze
- Reduktion von Netzwerknutzung und Ressourcenverbrauch durch Header-Komprimierung und effiziente Serialisierung
- Unterstützt eine Vielzahl von Programmiersprachen

Demo: Google RPC (gRPC)

Schnittstellendefinition: `hello.proto`

```
syntax = "proto3";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

Stub-Generierung

```
#!/bin/sh
rm -rf generated ; mkdir generated

python3 -m grpc_tools.protoc \
    -I. --python_out=generated --grpc_python_out=generated \
    hello.proto
```

Demo: Google RPC (gRPC)

Server: `server.py`

```python
import sys
sys.path.append("generated")

from concurrent import futures
import grpc
import hello_pb2
import hello_pb2_grpc

class Greeter(hello_pb2_grpc.GreeterServicer):
    def SayHello(self, request, context):
        return hello_pb2.HelloReply(message=f"Hello, {request.name}!")

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
    hello_pb2_grpc.add_GreeterServicer_to_server(Greeter(), server)
    server.add_insecure_port("[::]:50051")
    server.start()
    server.wait_for_termination()

if __name__ == "__main__":
    serve()
```

Demo: Google RPC (gRPC)

Client: `client.py`

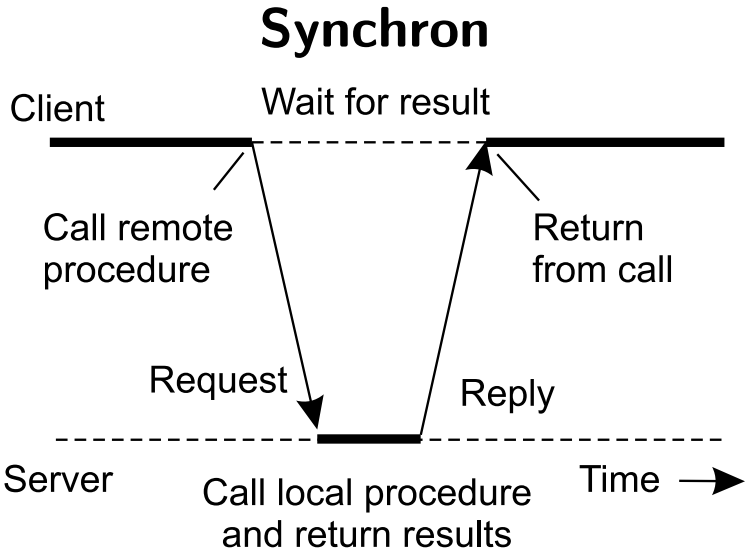
```python
import sys
sys.path.append("generated")

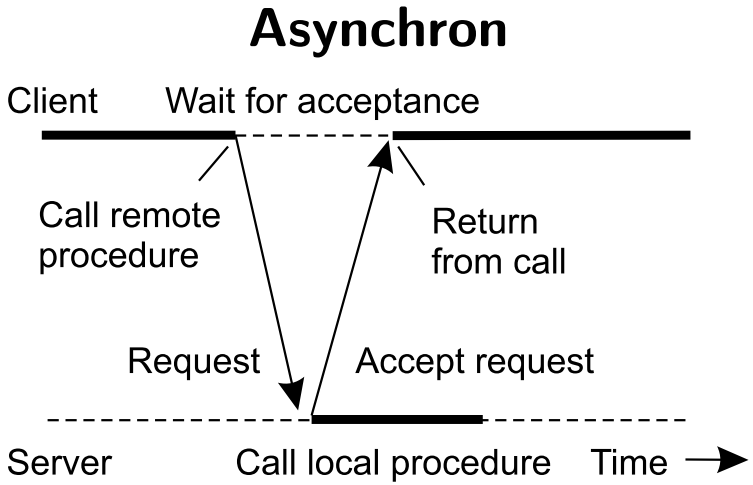
import grpc
import hello_pb2
import hello_pb2_grpc

def run():
    with grpc.insecure_channel("localhost:50051") as channel:
        stub = hello_pb2_grpc.GreeterStub(channel)
        response = stub.SayHello(hello_pb2.HelloRequest(name="Google"))
        print("Greeter client received: " + response.message)

if __name__ == "__main__":
    run()
```

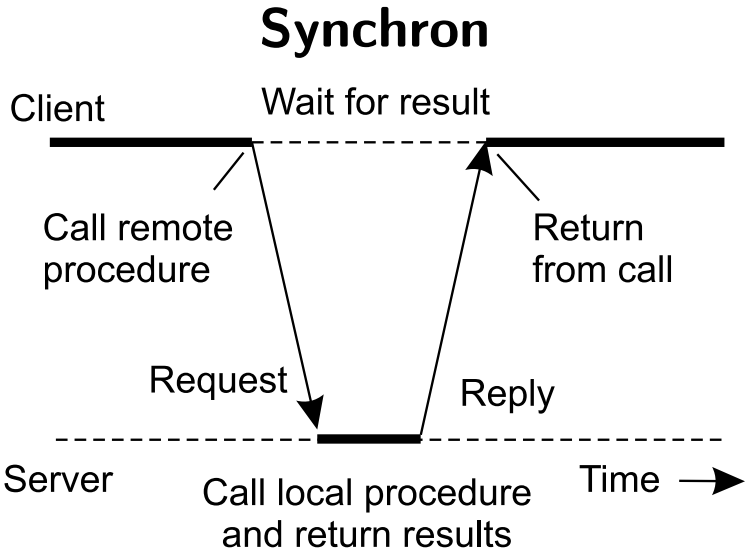
Synchrones vs. asynchrones RCP

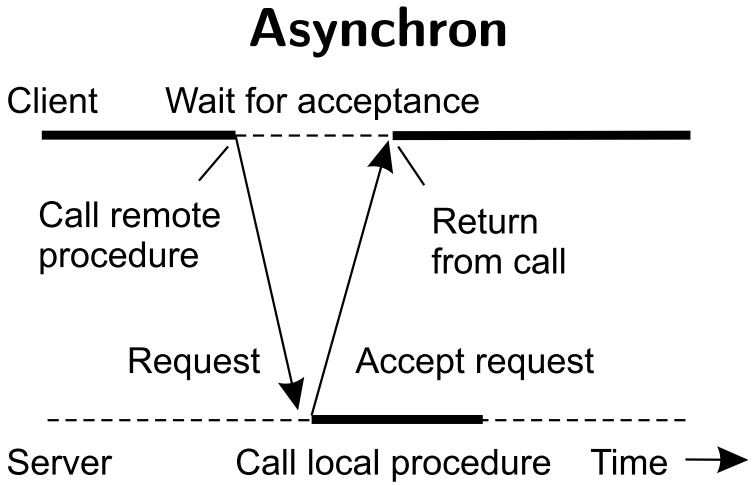




Wie erhält der Client beim asynchronen RCP das Ergebnis?

Synchrones vs. asynchrones RCP





Wie erhält der Client beim asynchronen RCP das Ergebnis?

- Client *pollt* Server $\rightsquigarrow$ wiederholtes, aktives Abfragen
- Server unterbricht Client $\rightsquigarrow$ RCP von Server zu Client

Inhalt

4. Kommunikation
- Protokolle
- Remote Procedure Call (RPC)
- Nachrichtenorientierte Kommunikation
- Datenströme
- Multicasting

Nachrichtenorientierte Kommunikation

Motivation

- Remote Procedure Calls (RPC) u. ä. verbergen Kommunikation in verteilten Systemen und ermöglichen Zugriffstransparenz
- Nachteile:
  - Synchrone Verarbeitung (zumeist) und Blockierung des Clients
  - Empfängerseite muss aktiv ausgeführt werden

Alternative

- Nachrichtenorientierte Kommunikation

$\rightsquigarrow$ Offenlegung der Kommunikation und Verzicht auf Zugriffstransparenz

Hauptaspekte

- Transiente nachrichtenorientierte Kommunikation
- Persistente nachrichtenorientierte Kommunikation

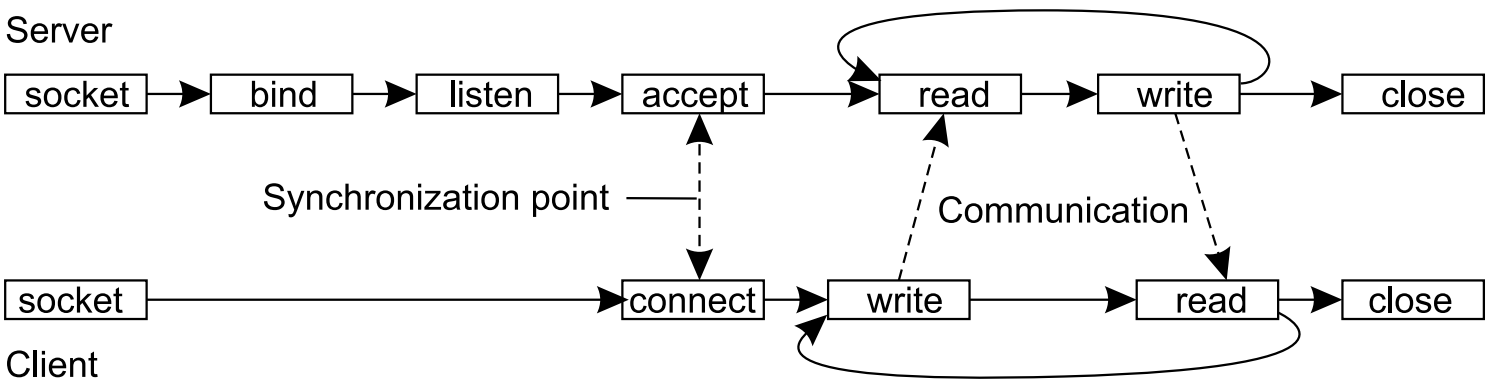
Transiente nachrichtenorientierte Kommunikation

- Nachrichten werden vom Kommunikationssystem solange gespeichert solange Kommunikationspartner ausgeführt werden

⇝ Sender und Empfänger müssen gleichzeitig ausgeführt werden

Beispiel 1: Berkely Sockets

- Standardisierung der Schnittstelle zur Transportschicht



Transiente nachrichtenorientierte Kommunikation

- Nachrichten werden vom Kommunikationssystem solange gespeichert solange Kommunikationspartner ausgeführt werden

$\rightsquigarrow$ Sender und Empfänger müssen gleichzeitig ausgeführt werden

Beispiel 2: Message Passing Interface (MPI)

- Kommunikationsstandard für *High Performance Computing* (HPC) Systeme
- Operationen für verschiedene Kommunikationsmuster
  - Punkt-zu-Punkt-Kommunikation: Send, Receive
  - Kollektive Kommunikation: Broadcast, Reduce, Gather, ...
  - Einseitige Kommunikation: Put, Get
- Vielzahl weiterer Service-Operationen

Demo: Message Passing Interface

Prozessinformationen: `hello.py`

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD        # Kommunikator für alle Prozesse
rank = comm.Get_rank()       # Prozess-ID (Rang)
size = comm.Get_size()       # Gesamtzahl der Prozesse

print(f"Hallo von Prozess {rank} von {size}")
```

Demo: Message Passing Interface

π-Berechnung: `pi.py`

```python
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

N = 10000000*5
steps_per_rank = N // size

start = rank * steps_per_rank
end = start + steps_per_rank

local_sum = 0.0
for i in range(start, end):
    x = (i + 0.5) / N
    local_sum += 4.0 / (1.0 + x*x)

total_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)

if rank == 0:
    pi = total_sum / N
    print(f"Berechneter Pi-Wert: {pi}")
```

Persistente nachrichtenorientierte Kommunikation

- Nachrichten werden vom Kommunikationssystem so lange wie nötigt gespeichert

⇝ Sender und Empfänger müssen nicht gleichzeitig ausgeführt werden

Nachrichtenwarteschlangensysteme

- Middleware für persistente, asynchrone Kommunikation über Warteschlangen
- Kernidee: Nachrichten werden in Warteschlangen gespeichert und weitergeleitet, auch wenn Sender oder Empfänger inaktiv sind

⇝ Kommunikation zwischen Sender und Empfänger ist zeitlich entkoppelt

Beispiele

- Internet E-Mail-Systeme
- Apache Kafka, Java Message Service (JMS)
- Advanced Message Queuing Protocol (AMQP), RabbitMQ, ZeroMQ, ...
- Robot Operating System (ROS)

Eigenschaften von Nachrichtenwarteschlangensystemen

Kommunikationsmodell

- Jede Anwendung hat (zumeist) eine eigene private Warteschlange
- Nachrichten werden über eindeutige Warteschlangennamen adressiert

Semantik

- Nur das Einfügen von Nachrichten in die Warteschlange wird garantiert
- Zeitpunkt oder Lesen der Nachricht ist nicht garantiert

Ausführungszenarien

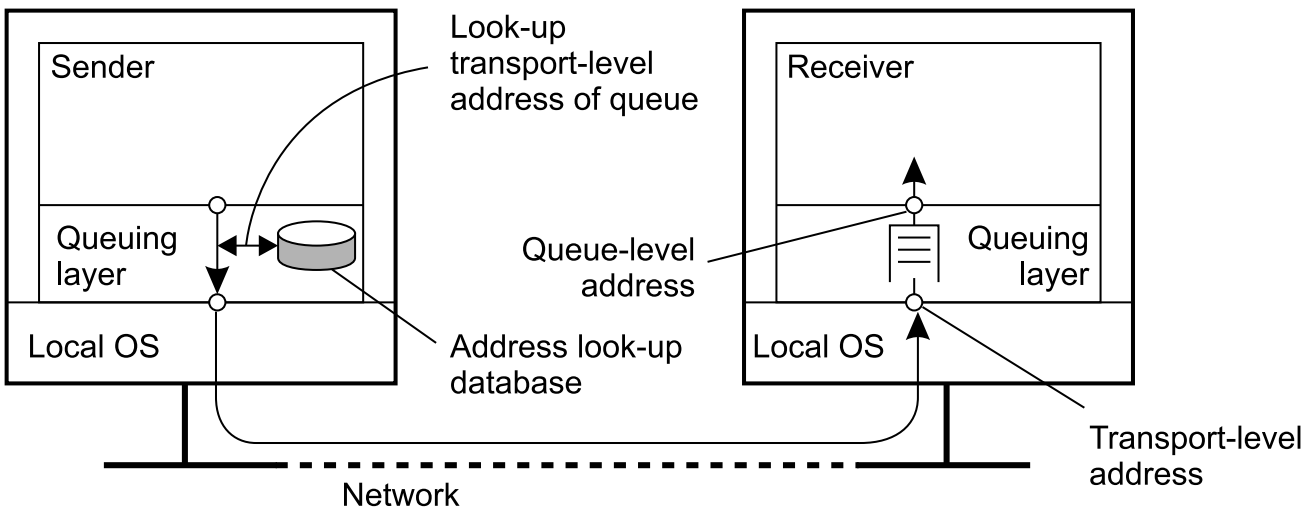
- Sender und Empfänger aktiv während Nachrichtentransfer
- Nur Sender oder nur Empfänger aktiv
- Sender und Empfänger passiv $\rightsquigarrow$ Nachrichten bleiben gespeichert

Operationen und Mechanismen

- Put $\rightsquigarrow$ Nachricht nicht-blockierend in Warteschlange einfügen
- Get $\rightsquigarrow$ blockierender Aufruf zum Lesen von Nachrichten
- Poll $\rightsquigarrow$ nicht-blockierendes Abfragen von Nachrichten
- Notify $\rightsquigarrow$ Automatische Nachrichtenbearbeitung durch registrierte Funktionen

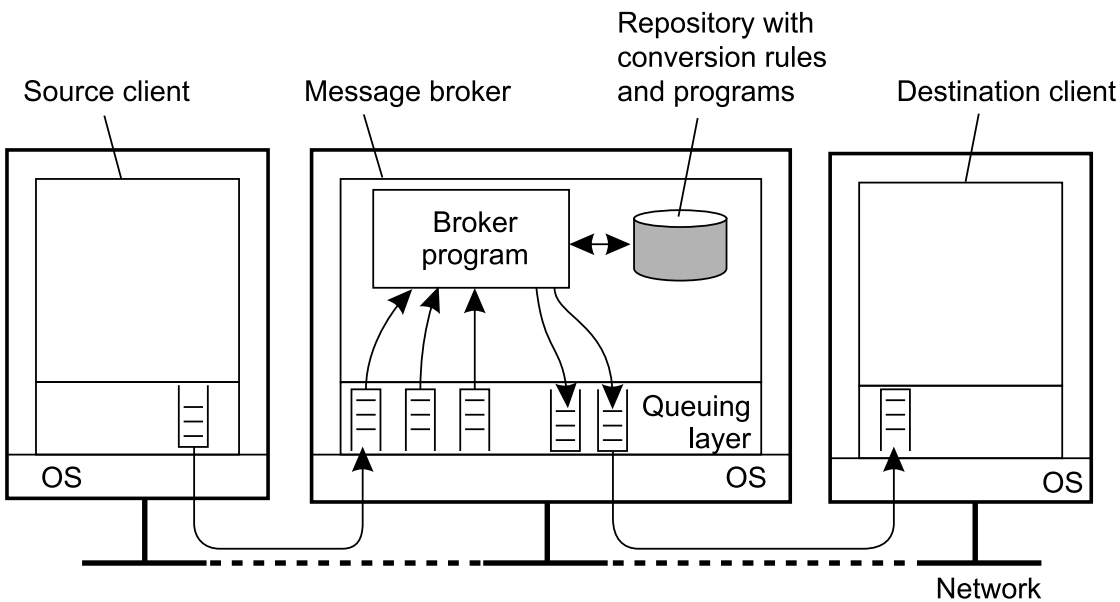
Architektur von Nachrichtenwarteschlangensystemen

- Anwendungen können nur auf lokale Warteschlangen zugreifen
- Ausgangswarteschlange nimmt zu versendende Nachrichten auf
- Zielwarteschlange enthält empfangen Nachrichten zur Entnahme
- ⇝ Middleware überträgt Nachrichten zwischen Warteschlangen
- Wartenschlagenmanager interagieren direkt mit Anwendungen
- Relays agieren als Zwischenstationen zur besseren Skalierung



Message Broker

- Hauptziel ist Integration vielfältiger Anwendungen in ein verteiltes System
- Nachrichtenkonvertierung notwendig, wenn Anwendungen sich nicht kennen
- Message Broker $\rightsquigarrow$ spezialisierter Knoten, der Nachrichten auf Anwendungsebene zwischen verschiedene Formaten übersetzen



Inhalt

4. Kommunikation
- Protokolle
- Remote Procedure Call (RPC)
- Nachrichtenorientierte Kommunikation
- Datenströme
- Multicasting

Datenströme

Motivation

- Bisher: Austausch von unabhängigen Informationseinheiten (Prozeduraufrufen, Nachrichten)
- Zeitunabhängigkeit $\rightsquigarrow$ Korrektheit der Kommunikation ist unabhängig vom genauen Zeitpunkt der Übertragung
- Neu: Zeitkritische Kommunikation $\rightsquigarrow$ Einhaltung genauer Zeitintervalle ist entscheidend

Beispiel

- Audiodatenstrom: 16-Bit Samples, 44,100 Hz, Stereo $\rightsquigarrow$ 176,4 KB/s
- Wiedergabe erfordert korrekte Reihenfolge und exakte Dauer von $\frac{1}{44{,}100}$ Sekunden pro Sample

Datenströme in verteilten Systemen

Definition

- Datenströme sind Folgen von Daten
- Diskrete Datenströme
  - Zeitpunkt der Übertragung ist nicht relevant für Interpretation der Daten
  - Beispiele: UNIX Pipes, TCP/IP-Verbindungen
- Kontinuierliche Datenströme
  - Zeitgenaue Übertragung ist notwendig für für Interpretation der Daten
  - Beispiele: Audio- und Videodaten
- Komplexe Datenströme
  - Mehrere zeitlich synchronisierte Unterdatenströme
  - Beispiele: Stereo-Audio, Audio und Video, Film mit Untertitel

Übertragungsmodi

- Asynchron $\rightsquigarrow$ keine zeitlichen Vorgaben, z. B. Dateiübertragung
- Synchron $\rightsquigarrow$ maximale End-to-End-Verzögerung gewährleistet
- Isochron $\rightsquigarrow$ strikte zeitliche Einhaltung (min./max. Verzögerung)

Streams und Servicequalität

- *Internet Protocol* (IP) Kommunikation ist nur eine *Best-Effort-Paketübertragung*

⇝ Paket können verworfen werden, wenn es „notwendig“ ist

Wie können bestimmte Servicequalitäten von Datenströmen sichergestellt werden?

Streams und Servicequalität

- *Internet Protocol* (IP) Kommunikation ist nur eine *Best-Effort-Paketübertragung*

⇝ Paket können verworfen werden, wenn es „notwendig“ ist

Wie können bestimmte Servicequalitäten von Datenströmen sichergestellt werden?

Netzwerkebene

- Klassifizierung und Priorisierung des Netzwerkverkehrs, z. B. *Differentiated Services Code Point* (DSCP), *Traffic Shaping*, *Network Slicing*
- *Content Delivery Networks* (CDN) ⇝ Datenquelle näher zum Nutzer bringen

Anwendungsebene

- *Adaptive Bitrate Streaming* (ABS) ⇝ Dynamische Anpassung der Qualität an verfügbare Netzwerkbandbreite
- *Forward Error Correction* (FEC) ⇝ Fehlerkorrektur mittels redundanter Zusatzinformationen
- Datenpufferung & verschachtelte Datenübertragung

Servicequalität durch Datenpufferung

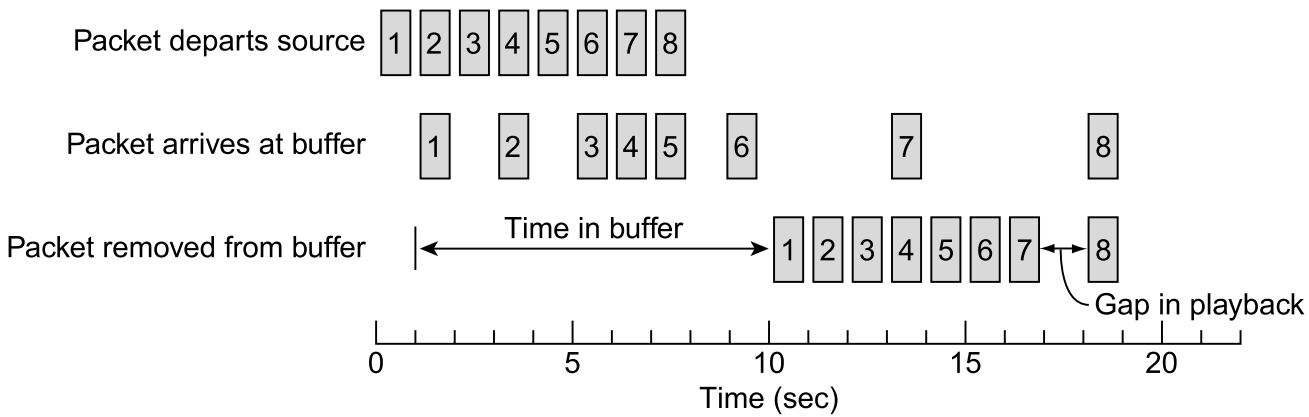
Grundidee

- Pakete werden in Puffer gespeichert, bevor sie weitergegeben werden

+ Übertragunsverzögerungen werden durch Paket im Puffer ausgeglichen

⇝ Verfügbarkeit der Pakete am Anfang des Datenstroms wird verzögert

Beispiel



Servicequalität durch verschachtelte Datenübertragung

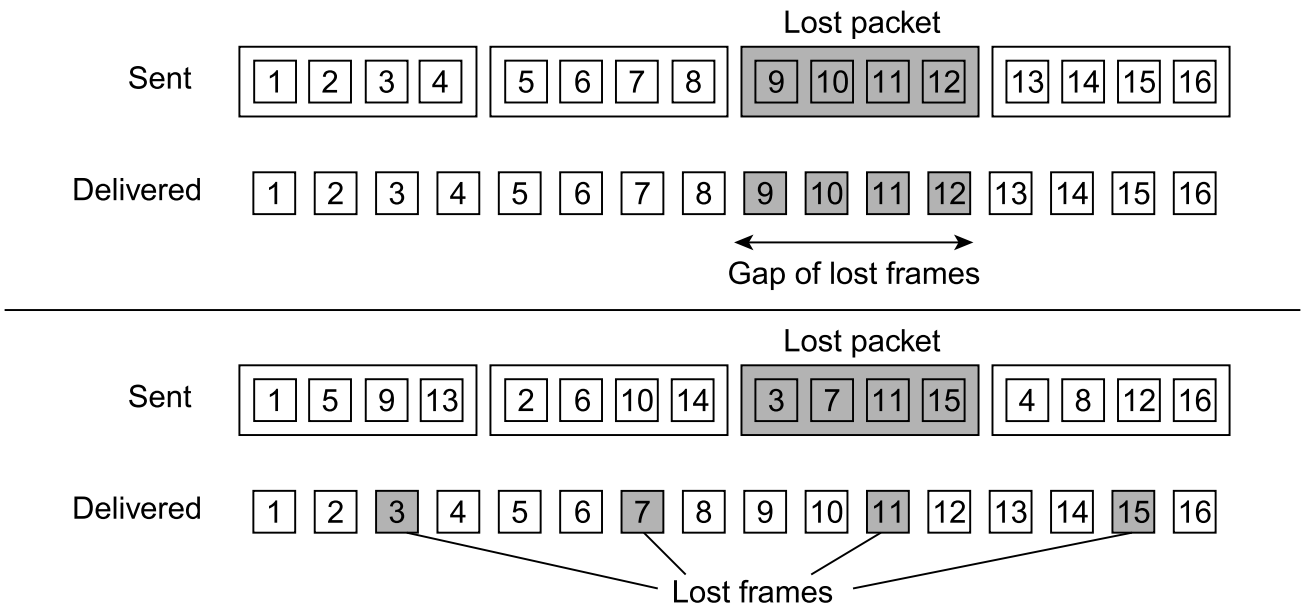
Grundidee

- Übertragungsreihenfolge der Pakete wird von Paketreihenfolge entkoppelt

+ Lücken durch Paketverluste werden zeitlich verteilt

⇝ Verfügbarkeit der Pakete am Anfang des Datenstroms wird verzögert

Beispiel



Inhalt

4. Kommunikation
- Protokolle
- Remote Procedure Call (RPC)
- Nachrichtenorientierte Kommunikation
- Datenströme
- Multicasting

Multicasting

Grundidee

- Kommunikation von einem Sender an mehrere Empfänger

⇝ Umsetzung durch paarweise Kommunikation zw. Sender und jedem Empfänger zu aufwändig

Umsetzungen auf Netzwerkebene

- Ethernet- und IP-Multicast ist in lokalen Netzwerken (LANs) möglich
- Multicast-Routing-Protokolle in Unternehmensnetzwerken verfügbar
- Komplexe Verwaltung und fehlende Unterstützung durch Internet-Service-Provider (ISP) ⇝ nur eingeschränkt im Internet verfügbar

Umsetzungen auf Anwendungsebene

- Nutzung von Peer-to-Peer-Technologien und Overlay-Netzwerken zur Einrichtung von Kommunikationspfaden
- Einsatz von Publish-Subscribe oder sog. Gossip-Protokollen

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz

Inhalt

1. Einführung
2. Architekturen
3. Prozesse
4. Kommunikation
5. Synchronisation
6. Konsistenz und Replikation
7. Fehlertoleranz